

Group 19 - X9v2 Resubmission Report

Members: Loo Zhi Yi, Sean Elisha Koh Tze Li

Design Choices

Architecture

X9v2 uses Jetpack Compose and is split into 2 Activities. `LoginActivity` uses FirebaseUI's pre-built sign-in screen since it already supports multiple sign-in methods (Google and email) and fulfills our requirements without the overhead of building a custom login. `MainActivity` hosts every other screen through a `NavHost` with a `NavController`. All route strings are stored in a `NavRoutes` object so that mistyped route strings cause compile-time errors rather than silent navigation failures.

We used Model-View-ViewModel (MVVM) because it separates the UI from the data logic, making it easier to manage state and test each layer independently. The app only deals with one data type (traffic reports) so a single `ReportViewModel` handles all the state. It exposes a `StateFlow<ReportUiState>` that Compose screens collect to update the UI whenever the data changes. A `ValueEventListener` attached in `init` listens for changes in the Firebase Realtime Database and parses each `DataSnapshot` child using `getValue(TrafficReport::class.java)`. The listener is removed in `onCleared()` to prevent memory leaks.

Data access is split into 2 repository classes: `ReportRepository` for database operations and `StorageRepository` for image uploads and deletes. This keeps database and file storage concerns in separate classes. The `ViewModel` creates both directly.

On startup, `X9Application` calls `setPersistenceEnabled(true)` so the app can read cached data when offline. It also loads `DATABASE_URL` from `assets/env` using `dotenv-kotlin` to keep the URL out of version control. Lastly, it registers Coil's `OkHttp` fetcher to load Firebase Storage images.

Data classes

`TrafficReport` is annotated with `@IgnoreExtraProperties` so that Firebase does not crash if the database contains fields the class does not define. Every field has a default value because Firebase's `getValue()` needs a no-arg constructor to deserialize snapshots. Severity is stored as an `Int` (1–5) instead of an enum since Firebase handles primitive types natively and an enum would require a custom serializer.

```
@IgnoreExtraProperties
data class TrafficReport(
    val type: String = "",
    val description: String = "",
    val severity: Int = 1,
    val latitude: Double = 0.0,
    val longitude: Double = 0.0,
    val creatorId: String = "",
    val userName: String = "",
    val imageUrl: String = "",
    val createdAt: Long? = null,
    val updatedAt: Long? = null,
)
```

The `ViewModel` also defines 2 supporting data classes. `ReportEntry` pairs a `TrafficReport` with its Firebase database key so that reports can be identified for updates and deletes. `ReportUiState` holds the list of `ReportEntry` objects that the UI screens observe through `StateFlow`.

User Interfaces

The app has 6 screens (see Appendix for screenshots). Navigation uses a bottom bar with 5 tabs: Home, Reports, Add, Map and Profile.

Dashboard (Figure 1): Shows a time-based greeting with the user's display name and a total report count. 2 quick action buttons ("New Report" and "View Map") let users go directly to report creation or the map. A Recent Reports section displays the 3 most recent reports as tappable cards. A "View All" button appears when there are more than 3 reports.

Report List (Figure 2): A `LazyColumn` of report cards showing type, severity, address, description preview, timestamp and an optional photo thumbnail loaded with Coil's `AsyncImage`. Filter chips at the top narrow down by traffic type with a count like "3 of 12 reports". Swiping is only enabled on reports the user created. Left-to-right opens the edit form while right-to-left triggers delete.

Report Form (Figure 3): Handles both creating and editing reports. An `ExposedDropDownMenuBox` picks the traffic type. An `OutlinedTextField` takes the description with validation. A `Slider` sets severity from 1 to 5 with colour-coded labels. 2 buttons attach a photo via camera (`TakePicture + FileProvider`) or gallery (`GetContent`). GPS coordinates come from `LocationService` and are reverse geocoded into an address. All fields use `rememberSaveable` to survive rotation. A `BackHandler` shows a discard dialog if there is unsaved input.

Report Detail (Figures 4–5): Shows the full report with sections separated by `HorizontalDivider`. Edit and Delete buttons only appear if the current user created the report, matching the same ownership rule as the swipe actions in the list.

Map (Figure 6): A `GoogleMap` composable with markers for every report that has coordinates. Tapping a marker shows an info card at the bottom with the report summary, timestamp, thumbnail and a "View Full Report" link. We used a bottom card instead of a map popup so there is room to show more detail. The selected marker uses `rememberSaveable` so it survives rotation. Location permission is requested at runtime with `ActivityResultContracts.RequestPermission()`.

Profile (Figure 7): Shows the user's display name and email. A `Switch` toggles dark mode. The preference is saved to `SharedPreferences` and applied through `X9ComposeTheme(darkTheme = isDarkMode)`. A sign-out button calls `AuthUI.getInstance().signOut()` and sends the user back to `LoginActivity`.

Extensions

1. **Dark mode:** A toggle on the profile screen switches between light and dark themes. The preference is saved to `SharedPreferences` so it persists across app restarts. We added this because the app could be used at night while driving.
2. **Filter chips:** The report list has chips for each traffic type, pulled from the `traffic_types` string-array. When a filter is active, the count updates to show how many reports match.

Without this, finding a specific type means scrolling through the entire list.

3. **Reverse geocoding:** All screens that display report locations run `Geocoder.getFromLocation()` on `Dispatchers.IO` to show a readable address instead of raw coordinates. We applied this on every screen that shows a location because raw latitude and longitude numbers are not useful to a user reading a traffic report.
 4. **Directional swipe actions:** Swiping left-to-right on a report card opens edit and right-to-left deletes it. Both directions show coloured backgrounds with icons. Swipe is disabled for reports the user did not create. This saves time compared to opening the detail screen to edit or delete.
 5. **Back-press discard guard:** Pressing back on the report form with unsaved input shows an `AlertDialog` asking whether to discard or keep editing. This prevents losing a partially filled form by accident.
 6. **Form state preservation:** All form fields (selected type, description, severity slider) use `rememberSaveable` so that rotating the device does not lose progress.
-

Testing and Evaluation

The project has 20 JUnit unit tests in `TrafficReportTest.kt` covering the `TrafficReport` data class and common list operations. These include default field values, `copy()` preserving unchanged fields, filter-by-type logic, sort-by-timestamp ordering, location presence checks, severity range validation, `take(3)` for dashboard reports, owner permission checks, description trimming and blank rejection, and image URL presence checks.

We also did manual smoke testing after each feature was added: form validation, full navigation flow, swipe-to-delete, camera and gallery attachment, dark mode persistence, rotation handling, filter chips, reverse geocoding and the back-press discard guard.

Testing was done on both a physical device and the Android Studio emulator. The physical device tested real camera and GPS behaviour. The emulator was used for spoofing GPS locations to verify map markers and geocoded addresses.

Problems

1. Coil 3 network image loading

- **Problem:** Report photos uploaded to Firebase Storage were not rendering. `AsyncImage` showed blank space where the image should be. After debugging we found that Coil 3's auto-discovery of the `OkHttp` fetcher factory was not picking it up reliably.
- **Resolution:** We registered the fetcher manually by calling `SingletonImageLoader.setSafe` in `X9Application.onCreate()` with an explicit `OkHttpNetworkFetcherFactory()`. Images loaded correctly after this.

2. Camera photo quality

- **Problem:** Photos taken through the app were too low-resolution to be useful. We were using `TakePicturePreview()` which only returns a small thumbnail `Bitmap`, not the full image.
- **Resolution:** We switched to `TakePicture()` which saves a full-resolution photo to a file URI. This required adding a `FileProvider` declaration in the manifest and a `file_paths.xml` resource to expose the cache directory. Camera permission is requested at runtime with `ActivityResultContracts.RequestPermission()`.

3. Choosing a Service type for GPS collection

- **Problem:** The app needs to collect GPS coordinates in the background while the user fills out the report form. We initially considered a bound service, which would let the form call methods directly on the service object through a `LocalBinder` and `ServiceConnection`. However, a bound service requires implementing a `Binder` subclass, a `ServiceConnection` callback, and managing `bindService()/unbindService()` lifecycle calls. Since our form only needs to receive location updates as they arrive and does not need to call methods on the service, this was more complexity than necessary.
- **Resolution:** We went with a started foreground service as the simpler option. `LocationService` collects GPS updates via `FusedLocationProviderClient` and broadcasts them through `LocalBroadcastManager`. The report form registers a `BroadcastReceiver` in a `DisposableEffect` to receive updates and stops the service when the form closes. Given time constraints this was the most straightforward approach that met the requirements. The tradeoff is that a bound service would give tighter control through direct method calls, but that was not needed for one-way location broadcasts.

4. Geocoder returning null on emulator

- **Problem:** While testing on the emulator, `Geocoder.getFromLocation()` would return null or an empty list, which caused the app to crash. The emulator's geocoder backend did not always resolve coordinates into addresses reliably.
- **Resolution:** We wrapped every `Geocoder` call in a try-catch on `Dispatchers.IO` so that if it returns null, throws an exception, or returns an empty list, the app falls back to showing no address. This is applied on all screens that display a location. The app still functions normally without geocoding. The address line simply does not appear.

5. Deprecated LocalBroadcastManager

- **Problem:** `LocalBroadcastManager`, which we use in `LocationService` to send GPS updates to the report form, is deprecated. Android Studio flags every usage with a warning. The recommended replacements are `SharedFlow` or `LiveData`, but both would require restructuring the service layer and adding coroutine or lifecycle dependencies to the service.
- **Resolution:** Since the broadcast only travels within our own app and does not expose data to other apps, we decided to keep `LocalBroadcastManager` and suppress the warning with `@file:Suppress("DEPRECATION")`. The simpler implementation was worth the tradeoff given the limited scope of the broadcast.

Appendix: Screenshots

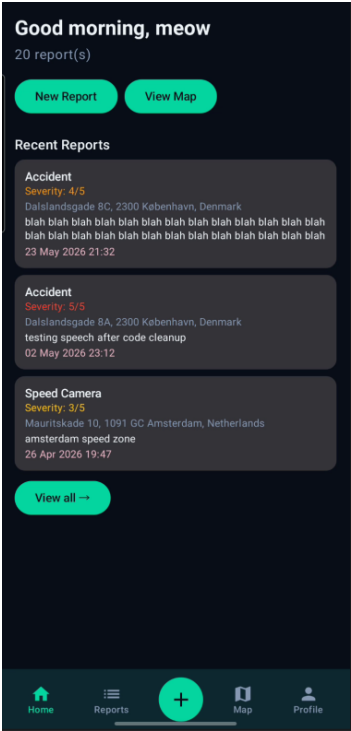


Figure 1: Dashboard

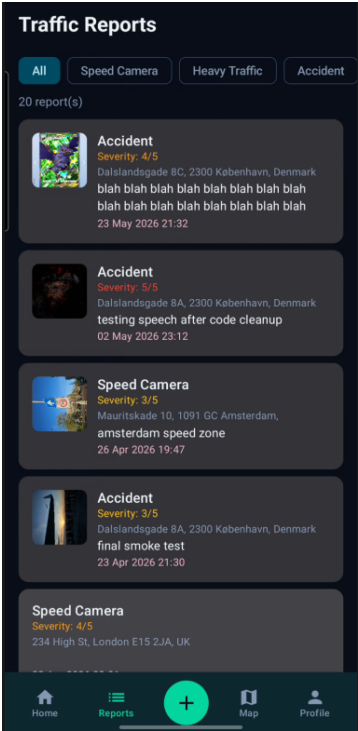


Figure 2: Report List

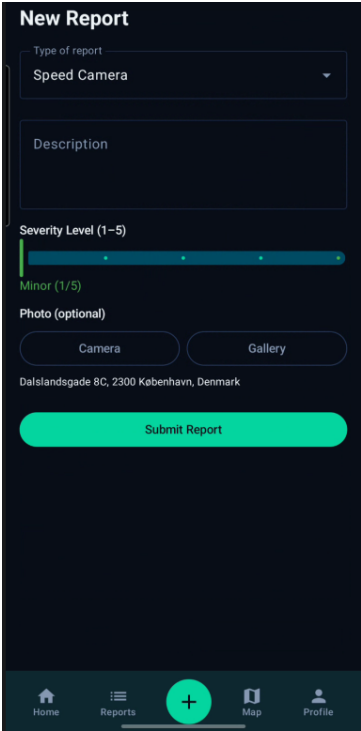


Figure 3: Report Form

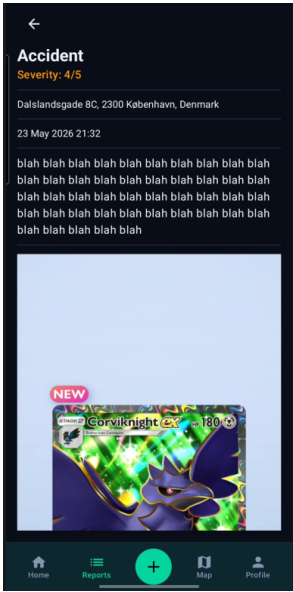


Figure 4: Detail

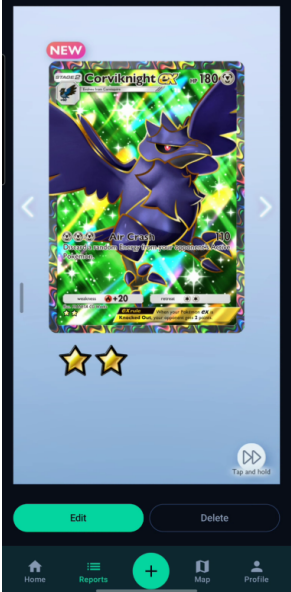


Figure 5: Detail (scrolled)

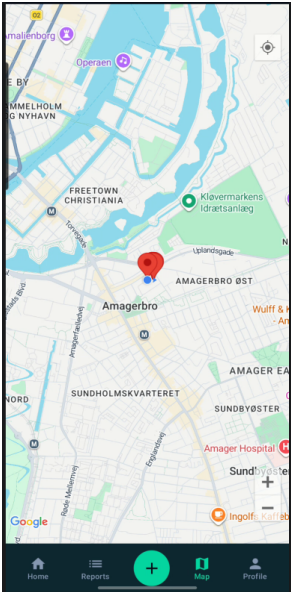


Figure 6: Map

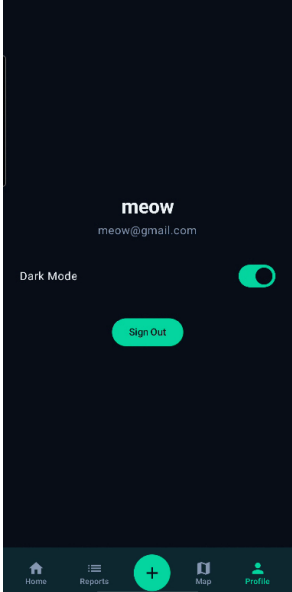


Figure 7: Profile